

FAST-MVH: Fast Adaptive Search with Trees for Multi-Valued Heuristic Search

Abstract

Multi-valued heuristics describe trade-offs that a single lower-bound vector cannot represent, but using them requires many dominance checks. FAST-MVH reduces this cost while preserving every search decision of $L\text{-NAMOA}_{dr}^*\text{-mvh}$. It indexes expensive truncated frontiers with adaptive k-d trees, tests local dominance before selecting a heuristic, and confines the exact full-dimensional fallback to a set of evicted vectors that we prove sufficient. That set is append-only and is indexed by a lazily built forest of static k-d trees. On grids and road networks with three to eight objectives, it returns byte-identical solutions. It runs 1.8 to 4.1 times faster than the reference implementation with three objectives and up to 82 times faster with six, and it solves an eight-objective grid with 1,082,326 Pareto-optimal costs in three minutes.

1 Introduction

A shortest path need not be best in every objective. One route may be shorter while another is faster. Multi-objective search must therefore retain paths with incomparable costs, rather than one cheapest path per state. As the retained sets grow, testing whether a path is dominated can cost more than generating it.

Multi-valued heuristics (MVHs) address a different difficulty: weak lower bounds. Instead of one estimate, an MVH gives several estimates of the remaining cost, each describing a possible trade-off. This information can reduce search, but using it is not free. A path may test several heuristic vectors before finding one that the current solutions do not refute.

Wolff, Felner, and Salzman [1] combine MVHs with dimensionality reduction in $L\text{-NAMOA}_{dr}^*\text{-mvh}$. Each path carries one active heuristic vector. If a new solution refutes it, the algorithm selects the next one and reinserts the path. Local dominance is tested on truncated vectors when that suffices and on full vectors otherwise.

FAST-MVH keeps this algorithm and reduces the work needed to make its decisions. It changes three things: how frontiers are stored, the order of the two tests applied to a generated path, and the set that the exact full-dimensional test must search. It does not change the heuristic, the ordering of OPEN, or the Pareto set returned. Indexing truncated frontiers with dynamic

3 FAST-MVH

Algorithm 1 follows the node lifecycle of $L\text{-NAMOA}_{dr}^*\text{-mvh}$ [1]: extract a node; if the goal frontier refutes its heuristic, select the next one and reinsert; otherwise test it locally, expand it, and generate successors. One step differs. At generation, the reference implementation selects a heuristic first and then tests local dominance. FAST-MVH tests local dominance first (lines 15–16).

k-d trees follows prior work on geometric dominance indexing [2]. The contributions here are its integration with lazy MVH evaluation and a provably sufficient, indexed fallback set.

2 Problem and Search Invariants

Let $G = (S, E, c)$ be a finite graph with $c(e) \in \mathbb{R}_{\geq 0}^M$. Path costs are component-wise sums. A vector x weakly dominates y , written $x \preceq y$, if $x_k \leq y_k$ for every k . The task is to return one path for each distinct non-dominated start-to-goal cost vector. Write $x <_{\text{lex}} y$ for lexicographic order and

$$\text{Tr}(x) = (x_2, \dots, x_M).$$

An admissible MVH assigns a finite set $H(s)$ to each state: for every goal-reaching suffix π from s , some $h \in H(s)$ satisfies $h \preceq c(\pi)$. We assume $H(s_{\text{goal}}) = \{0\}$ and the ordering used by [1]:

$$H(s) = \langle h^1, \dots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \dots \leq_{\text{lex}} h^{k(s)}.$$

For genuine MVHs this order is a correctness precondition, not a traversal preference: the selected vector must be the first surviving member. No index may change that answer.

Example. Suppose two suffixes cost $(1, 10)$ and $(10, 1)$. The set of these vectors is admissible, but its component-wise maximum $(10, 10)$ bounds neither suffix. If every vector bounded *every* suffix, as landmark bounds do, the maximum would be admissible. Genuine MVHs generally lack this property [3,4], so we never replace an MVH by its maximum.

A node n records a state $s(n)$, path cost $g(n)$, heuristic index $i(n)$, and $f(n) = g(n) + h^{i(n)}$. OPEN is ordered lexicographically by f . Let $C(s)$ be the costs of paths expanded at s . The *reduced frontier* $F(s) \subseteq C(s)$ stores full vectors but is compared on truncations. Expanding g at s performs

$$F(s) \leftarrow \{p \in F(s) : \text{Tr}(g) \not\preceq \text{Tr}(p)\} \cup \{g\}.$$

Let $T = F(s_{\text{goal}})$. $F(s)$ need not be a full-dimensional Pareto set. Its required property is *coverage*: every $a \in C(s)$ has some $p \in F(s)$ with $\text{Tr}(p) \preceq \text{Tr}(a)$.

FAST-MVH also keeps a *residue* $R(s) \subseteq C(s)$. When an expansion with cost g evicts p from $F(s)$, p enters $R(s)$ iff $p_1 < g_1$. Otherwise $g \preceq p$ in all M coordinates and p is discarded. Vectors never leave $R(s)$, and $C(s)$ itself is not stored.

Algorithm 1: FAST-MVH**Input:** $G, s_{\text{start}}, s_{\text{goal}}$ and ordered admissible H .**Output:** a cost-unique Pareto-optimal path set Sols.

```

1 Sols  $\leftarrow \emptyset$ ;  $F(s), R(s) \leftarrow \emptyset$  for all  $s$ 
2  $n \leftarrow (s_{\text{start}}, 0, 1)$ ;  $\text{parent}(n) \leftarrow \perp$ 
3 OPEN  $\leftarrow \{n\}$ 
4 while OPEN  $\neq \emptyset$  do
5    $n \leftarrow \text{OPEN.PopMin}()$ 
6   if GoalDom( $f(n)$ ) then
7      $i \leftarrow \text{ChooseH}(s(n), g(n), i(n) + 1, \text{true})$ 
8     if  $i \neq \perp$  then  $i(n) \leftarrow i$ ; insert  $n$  in OPEN
9     continue
10  if LocalDom( $s(n), g(n)$ ) then continue
11  Update( $s(n), g(n)$ )
12  if  $s(n) = s_{\text{goal}}$  then add  $n$  to Sols; continue
13  for each  $s' \in \text{Succ}(s(n))$  do
14     $g' \leftarrow g(n) + c(s(n), s')$ 
15    if LocalDom( $s', g'$ ) then continue
16     $i' \leftarrow \text{ChooseH}(s', g', 1, \text{false})$ 
17    if  $i' \neq \perp$  then insert  $(s', g', i')$  with parent  $n$  in OPEN
18 return Sols

```

The two tests at generation are pure: neither changes a frontier. Their order therefore cannot change which successors enter OPEN. It changes only cost. ChooseH may query the goal frontier once per heuristic vector, while LocalDom usually decides with one frontier query. Testing first the predicate that is cheaper and prunes more often avoids selecting heuristics for paths that would be discarded anyway.

3.1 Dominance queries

Every frontier is stored in a k-d tree [5]. A node v holds a few live vectors $P(v)$, at most two children, and the component-wise minimum $\ell(v)$ and maximum $u(v)$ of the vectors below it. An unindexed array is a single node. Find($\mathcal{K}, q, \kappa$) enumerates the members p of a tree or forest $\mathcal{K}$ with $\kappa(p) \leq q$, where κ is Tr or the identity. It skips a subtree when $\kappa(\ell(v)) \not\leq q$, because then no vector below v can qualify. Callers stop it at the first useful answer.

Algorithm 2: Dominance queries and update

```

1 function Find( $\mathcal{K}, q, \kappa$ )
2    $\Sigma \leftarrow \text{roots of } \mathcal{K}$ 
3   while  $\Sigma \neq \emptyset$  do
4     pop  $v$  from  $\Sigma$ ; if  $\kappa(\ell(v)) \not\leq q$  then continue
5     yield each  $p \in P(v)$  with  $\kappa(p) \leq q$ 
6     push the children of  $v$  on  $\Sigma$ 
7 function GoalDom( $f$ )
8   return true iff Find( $T, \text{Tr}(f), \text{Tr}$ ) yields
9 function LocalDom( $s, g$ )
10   $w \leftarrow \text{NONE}$ 
11  for each  $p \in \text{Find}(F(s), \text{Tr}(g), \text{Tr})$  do
12    if  $p_1 \leq g_1$  then return true  $\triangleright p \leq g$ 
13     $w \leftarrow \text{TR}$ 
14  if  $w = \text{NONE}$  then return false
15  return true iff Find( $R(s), g, \text{id}$ ) yields
16 procedure Update( $s, g$ )
17  for each  $p \in F(s)$  with  $\text{Tr}(g) \leq \text{Tr}(p)$  do
18    remove  $p$  from  $F(s)$ 
19    if  $p_1 < g_1$  then append  $p$  to  $R(s)$ 
20  insert  $g$  into  $F(s)$ 

```

GoalDom(f) asks whether some solution cost dominates f in its last $M - 1$ coordinates. LocalDom(s, g) asks whether some expanded path at s dominates g in all M coordinates. A single traversal of $F(s)$ returns one of three answers: no truncated dominator, a full dominator (line 12), or only truncated dominators ($w = \text{TR}$). Only the last case consults $R(s)$, in all M coordinates (line 15). Line 17 traverses the same tree symmetrically, skipping v when $\text{Tr}(g) \not\leq \text{Tr}(u(v))$.

The reference implementation decides the third case differently. It keeps $m(s) = \max_{p \in F(s)} p_1$, accepts when $m(s) \leq g_1$, and otherwise scans all of $\mathcal{C}(s)$. Line 12 makes the $m(s)$ test redundant: if $m(s) \leq g_1$, the first truncated dominator found is a full one.

3.2 Heuristic selection

ChooseH(s, g, j, b) returns the first index $i \geq j$ whose vector the goal frontier does not refute, or $\perp$. The redundancy pointer $\rho_s(i)$ is the largest $i' < i$ with $\text{Tr}(h^{i'}) \leq \text{Tr}(h^i)$, or 0 if none exists. It is computed once per state, when $H(s)$ is first used.

Algorithm 3: ChooseH(s, g, j, b)**Precondition:** b is true only if h^{j-1} was just refuted by GoalDom.

```

1 if  $j > k(s)$  then return  $\perp$ 
2 if  $T = \emptyset$  then return  $j$ 
3  $r \leftarrow j - 1$  if  $b$  else  $j$ 
4 for  $i = j, \dots, k(s)$  do
5   if  $\rho_s(i) \geq r$  then continue  $\triangleright$  refuted by  $h^{\rho_s(i)}$ 
6   if not GoalDom( $g + h^i$ ) then return  $i$ 
7 return  $\perp$ 

```

The truncated vectors of $H(s)$ are stored contiguously. Candidates are examined in the order of [1]; line 5 only skips

candidates known to be refuted, and omitting a pointer only loses a shortcut.

4 Why the Changes Are Safe

Lemma 1 (Residue sufficiency). Suppose no $p \in F(s)$ satisfies $p \preceq g$. Then some $a \in C(s)$ satisfies $a \preceq g$ iff some $r \in R(s)$ does.

Proof. $R(s) \subseteq C(s)$ gives one direction. For the other, let $a_0 \in C(s)$ with $a_0 \preceq g$. Since $a_0 \notin F(s)$ by assumption, some later expansion b evicted it, with $\text{Tr}(b) \preceq \text{Tr}(a_0)$. If $a_0 \in R(s)$ we are done. Otherwise $b_1 \leq (a_0)_1$, so $b \preceq a_0 \preceq g$; let $a_1 = b$ and repeat. Each step moves to a strictly later expansion, so the chain is finite. It cannot end in $F(s)$, by assumption. It therefore ends in $R(s)$. $\square$

Lemma 2 (Local exactness). $\text{LocalDom}(s, g)$ returns true iff some $a \in C(s)$ satisfies $a \preceq g$.

Proof. Coverage holds initially and is preserved by Update: the new member g truncation-dominates every vector it evicts, and $\preceq$ is transitive. If Find yields nothing, no $a \in C(s)$ dominates even $\text{Tr}(g)$, so false is correct. A return at line 12 exhibits $p \in F(s) \subseteq C(s)$ with $p \preceq g$. Otherwise the traversal has examined every truncated dominator in $F(s)$ and found none that is full, so Lemma 1 applies to line 15. Box pruning in Find discards only subtrees that contain no qualifying vector. $\square$

Example. Let $a = (3, 3, 3)$ be expanded at s , and later $b = (9, 2, 2)$. Update evicts a , since $\text{Tr}(b) \preceq \text{Tr}(a)$; as $3 < 9$, a enters $R(s)$. A new path $g = (4, 3, 3)$ has the truncated dominator b but no full dominator in $F(s)$. Line 15 finds a . Had a been $(9, 3, 3)$ and $b = (3, 2, 2)$, then $b \preceq a$, and any g dominated by a is also dominated by b , which line 12 finds; a need not be kept.

Lemma 3 (Safe skipping). Line 5 of Algorithm 3 skips only refuted candidates.

Proof. T changes only by Update, which preserves coverage. Hence a truncated query refuted by T stays refuted. Every index in $[r, i)$ has been refuted, either during this call or, when b holds, by the goal test that preceded it. If $\rho_s(i) \geq r$, then $\text{Tr}(g + h^{\rho_s(i)}) \preceq \text{Tr}(g + h^i)$, so the dominator that refuted $h^{\rho_s(i)}$ also refutes h^i . $\square$

Theorem 1 (Decision equivalence). Given identical heuristic arrays, successor order, arithmetic, and OPEN tie-breaking, FAST-MVH and the reference implementation of [1] make the same OPEN insertions and extractions and return the same solutions in the same order.

Proof. Induct on extractions. Both searches start alike. GoalDom is an exact query on the same T . Lemma 2 gives the reference’s local answer. By Lemma 3, ChooseH returns the first surviving index, as the reference does. Both searches therefore expand the same paths and keep the same live members of $F(s)$. At generation the two tests are pure, so exchanging them preserves both their conjunction and the selected index. The same nodes enter OPEN in the same order. $\square$

Under the assumptions of [1], this equivalence preserves completeness and Pareto optimality. It does not repair an inadmissible heuristic or an invalid order.

5 Representation and Cost

Adaptive frontiers. Each $F(s)$ starts as a contiguous array. A frontier of at least eight vectors whose scans average 64 or more comparisons per query is promoted to a dynamic k-d tree over its last $M - 1$ coordinates. Nodes live in an index-addressed pool and store lower and upper corners. Deletions mark nodes inactive; the tree is rebuilt when its size exceeds $1.25b + 64$, where b is the live count at the last build. These parameters affect cost, not correctness.

The residue forest. $R(s)$ is append-only, so it suits the logarithmic method of Bentley and Saxe [6]. New vectors are appended to an unindexed buffer. Nothing is built until a query finds 64 buffered vectors. The buffer then becomes a static implicit k-d tree: a contiguous block permuted so that each median sits at the middle of its range, with axes cycling through all M coordinates and a lower corner per subtree. A block is merged with its predecessor while the predecessor is less than twice its size, so a state holds $O(\log|R(s)|)$ blocks and each vector is rebuilt $O(\log|R(s)|)$ times. A query scans the buffer and then searches the blocks. States that never fall back never build a tree.

$R(s)$ omits two kinds of vector that the reference fallback scans: the live members of $F(s)$, which line 11 has already examined, and evicted vectors that their evictor dominates fully. Most of the saving, however, comes from the index (Section 6.3).

Remark (prior design). An earlier version of this fallback cached, per candidate, one goal vector that had previously refuted it, so a repeated candidate could often be refuted at once without a scan. This was a real gain over scanning $C(s)$ unindexed, roughly 10-18% faster wall time on the largest instances in our records. It is a heuristic, however: a cache miss still falls back to a linear scan, and the scan itself is not made any smaller. The indexed residue forest above replaces it, since every fallback query is now exact and sub-linear regardless of cache state, so the extra cache no longer earns its keep and is not part of the method evaluated here.

Why not index every heuristic set? A frontier box test compares a vector with a corner. A box test over $H(s)$ asks whether a corner is dominated by the *goal frontier*, which is itself a full query. For a subtree with truncated corners ℓ, u , an index may reject all members if $\text{GoalDom}(g + \ell)$ and accept the first member if $\neg \text{GoalDom}(g + u)$. Both rules are sound but often decide nothing (Figure 1). Local-first testing removes most selection calls, and redundancy pointers remove many of the rest. The evaluated configuration therefore leaves this index disabled.

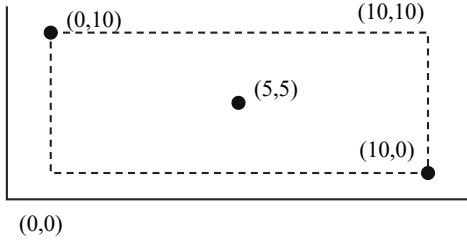


Figure 1. A heuristic box contains unattainable corners. For $g = 0$, goals at the three members refute all members but not the lower corner. A goal at $(6,6)$ refutes the upper corner but no member.

OPEN. Entries hold f inline with the index of a record that stores g , the state, and the parent. The comparator is the reference’s lexicographic order on f , so the extraction sequence is unchanged; solution paths are materialized once, at the end.

6 Experimental Results

We compare three solvers on one machine (Intel Core i5-1135G7, 16 GB, Windows 11). All are compiled by GCC 16.2 with `-O2 -DNDEBUG` under C++20. The *reference* is the

implementation of [1] (unmodified source, commit 0a2f9ea), driven by a harness that writes its solutions. FAST_C is our earlier configuration. It shares FAST-MVH ’s frontier trees, test order, and heuristic selection, but scans all of $\mathcal{C}(s)$ on fallback (behind the $m(s)$ test) and stores nodes by pointer. Times are search times, excluding parsing. FAST-MVH and FAST_C times are minima of three interleaved runs; the reference runs once, or three times when it takes under 5 s. In all 27 instances, every solver that finished wrote a byte-identical solution file and reported the same numbers of solutions, expansions, and extractions.

Instances. Grids are four-connected $n \times n$ graphs whose integer costs have pairwise correlation $\rho \in [-0.6, 0]$. Roads are breadth-first subgraphs of the DIMACS New York and Bay Area networks [8]. $\text{NY-}k$ and $\text{Bay-}k$ have k thousand vertices, and the centre is the goal. The first two objectives are distance and time; the others are uniform integers in $[1, 100]$. Heuristics are A*pex backward Pareto sets [7] with approximation ε , sorted lexicographically as [1] specifies. Several archived files listed ties or whole sets out of this order, and we sorted them. Where we ran both versions, the solution files were unchanged and expansions differed by at most 0.04%.

Table 1. Three to six objectives. Times in seconds; speedup is reference time divided by FAST-MVH time. The reference was stopped at 600 s; the percentage is the fraction of expansions it reached. Single runs in the last four rows.

Instance	M	ε	$ \Pi $	Expansions	Reference	FAST_C	FAST-MVH	Speedup
Grid 10	3	0	1,602	9,500	0.058	0.044	0.032	1.8
Bay-8	3	0.1	238	12,346	0.053	0.040	0.018	2.9
Bay-16	3	0.1	3,534	347,498	2.67	1.76	0.647	4.1
Grid 10	4	0.05	11,330	53,652	1.74	0.460	0.338	5.1
NY-5	4	0.01	7,105	144,117	9.55	1.85	1.04	9.2
Bay-8	4	0.1	3,716	157,704	20.1	1.32	0.654	30.7
NY-8	4	0.05	5,713	195,594	39.1	1.91	1.03	37.9
Grid 8	5	0.05	9,412	33,005	1.47	0.289	0.241	6.1
NY-5	5	0.05	26,701	503,505	184	8.84	5.46	33.7
Grid 8	6	0.05	70,196	179,297	92.0	2.97	2.34	39.4
NY-3	6	0.05	11,286	127,349	104	1.57	1.27	82.0
Bay-16	4	0.1	55,069	2,881,236	>600 (85%)	41.8	21.9	>27
Bay-8	5	0.1	33,104	1,195,876	>600 (39%)	27.8	19.1	>31
NY-8	5	0.1	57,902	2,049,014	not run	49.5	32.2	—
NY-5	6	0.1	184,366	2,642,117	not run	87.2	63.7	—

6.1 Three to six objectives

At three objectives FAST-MVH is 1.8–4.1 times faster than the reference. From four objectives on, every road case gains at least 9.2 times, and the six-objective NY case gains 82 times. On Bay-8 in 5D, the reference reached 39% of the expansions in 600 s; FAST-MVH finished in 19 s. The grids gain less than roads of similar output: Grid 8 in 5D returns more costs than NY-5 in 4D, yet gains 6.1 against 9.2. Neither dimension nor output size alone predicts the gain (Figure 2).

Three objectives. Truncation leaves two coordinates. Frontier queries are short scans, and Grid 10 promotes no frontier to a tree. The gain over FAST_C is largest here (1.4–2.7 times), although fallbacks are rare: on Bay-16, FAST_C spends 1.1 million full comparisons in 1.8 s. Node storage dominates at this scale, and inline OPEN keys reduce it. Bay-16 still promotes 11 frontiers, so the representation follows scan cost, not dimension.

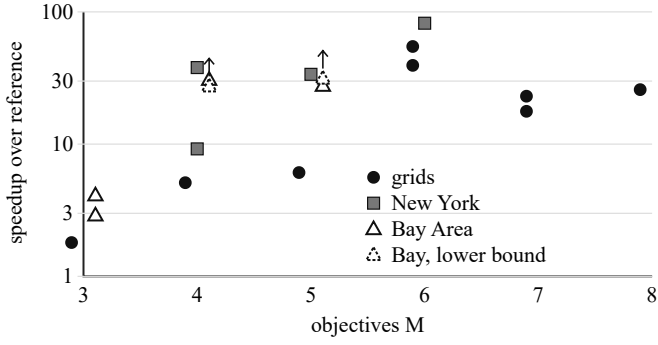


Figure 2. Speedup of FAST-MVH over the reference on one machine (log scale), from Tables 1 and 2. Dashed triangles are lower bounds where the reference was stopped at 600 s.

Table 2. Five to eight objectives. Grids use $\varepsilon = 0.05$; Bay-0.8 is an 800-vertex Bay subgraph with $\varepsilon = 0.1$. Times in seconds; “—” means not run. In the last seven rows the reference time (*italics*) is from the second machine, and the speedup is the bound described in the text.

Instance	M	$ \Pi $	Expansions	Reference	FAST _C	FAST-MVH	Speedup
Bay-0.8	5	7,617	34,871	5.98	0.245	0.216	27.7
Grid 8, $\rho = -0.2$	6	104,480	293,896	321	7.43	5.85	54.8
Grid 7, $\rho = -0.2$	7	32,643	91,704	24.5	1.50	1.38	17.7
Grid 7, $\rho = -0.4$	7	49,884	132,959	57.6	2.83	2.51	23.0
Grid 6, $\rho = -0.2$	8	47,079	99,244	37.0	1.87	1.44	25.8
Bay-0.8	6	94,886	351,954	3,775	7.84	6.09	≥ 180
Grid 8, $\rho = -0.2$	7	124,943	373,533	<i>1,514</i>	11.8	9.95	≥ 44
Grid 9, $\rho = -0.4$	7	266,224	815,838	<i>3,430</i>	32.2	20.2	≥ 49
Grid 9, $\rho = 0$	7	416,597	1,305,173	<i>6,626</i>	—	43.4	≥ 44
Grid 7, $\rho = -0.2$	8	175,239	445,827	<i>1,006</i>	20.7	12.1	≥ 24
Grid 8, $\rho = -0.4$	8	366,205	1,033,696	<i>8,553</i>	—	40.5	≥ 61
Grid 8, $\rho = -0.2$	8	1,082,326	3,092,094	<i>>35,013</i>	—	183	≥ 56

The largest case has 1,082,326 Pareto-optimal costs. On the second machine the reference exhausted memory after 9.7 hours; here FAST-MVH finishes in about three minutes. The measured speedups at seven and eight objectives (18–26) are smaller than the one at six (55). As in Table 1, the gain follows the frontier workload of an instance, not M .

6.3 The fallback

Table 3 isolates the residue forest. Across the 24 cases where both FAST_C and FAST-MVH finished, it reduces full comparisons by 8 to 100 times in every case with more than 1,000 fallbacks. $R(s)$ itself is not small: it holds up to 93% of

6.2 Up to eight objectives

The instances in Table 2 were built for an earlier campaign on a second machine. We reran all three solvers here; the reference completed the five smallest within six minutes. For the seven others we cite its time on the second machine, which used unsorted heuristic files and a different build. On the five common instances, the reference’s time ratio between the machines ranged from 0.29 to 1.49. We therefore compute no speedup from cross-machine times. Instead we give a conservative bound: the second-machine time multiplied by 0.29, divided by FAST-MVH’s time here.

the expanded vectors. The reduction therefore comes mainly from indexing, as Section 5 anticipated. The time ratio FAST_C/FAST-MVH ranges from 1.09 to 2.73. Because it also includes the node-storage change, it bounds the fallback’s contribution from above.

Fallback counts depend on the instance more than on M . Bay-0.8 in 6D needs only 80 fallbacks in 351,954 expansions; NY-5 in 6D needs 185,455. Local-first ordering can raise the count. On Grid 10 in 3D, FAST-MVH makes 3,806 fallback queries and the reference 2,468: some successors that the reference rejects during selection reach the local test first. Theorem 1 preserves decisions, not the number of calls to each test.

Table 3. Full-dimensional fallback. Fallbacks counts third-case queries. Comparisons are full-vector comparisons, in millions. $|R|$ is the final total residue size, divided by expansions.

Instance	M	Fallbacks	Comparisons, FAST _c	Comparisons, FAST-MVH	$ R /\text{Exp.}$	Time ratio
Bay-16	3	5,783	1.14	0.144	0.67	2.73
Bay-16	4	52,298	87.7	2.36	0.66	1.91
NY-5	5	91,934	93.2	4.83	0.70	1.62
NY-5	6	185,455	1,097	13.3	0.50	1.37
Grid 8	6	14,804	81.2	1.86	0.64	1.27
Grid 9, $\rho = -0.4$	7	101,098	1,957	20.9	0.71	1.60
Grid 7, $\rho = -0.2$	8	74,047	1,356	13.6	0.65	1.71

Reproducibility. Runs are in benchmarks/runs/ under the suffixes fast_m3_6, fast_ny_lex, and fast_m5_8_home_instances. Each run directory holds the commands, commit, binary hashes, solution files, and every counter. The Bay instances share one centre, and the extra road objectives are synthetic, so the instances are not independent samples of routing problems.

trees on truncated frontiers, local tests before heuristic selection, and an exact fallback confined to a provably sufficient residue indexed by a k-d forest. On one machine, across grids and two road networks with three to eight objectives, it returned identical solutions and ran 1.8 to 82 times faster than the reference. The gain followed frontier workload rather than dimension. Non-lexicographic MVH search requires separate ordering arguments [4] and is left for future work.

7 Conclusion

FAST-MVH makes the same decisions as $L\text{-NAMOA}_{dr}^*\text{-mvh}$ at lower cost. Three changes account for this: adaptive k-d

References

1. Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. *Proceedings of the Symposium on Combinatorial Search*. Reference manuscript in the project archive.
2. Anonymous. *A Geometric Index for Multi-Objective Dominance Checking*. Unpublished manuscript in the project archive, associated there with S. S. Shperberg.
3. Geisser, F.; Haslum, P.; Thiebaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. *Proceedings of ICAPS*, 100–109.
4. Skyler, S.; Shperberg, S. S.; Atzmon, D.; Felner, A.; Salzman, O.; Chan, S.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernandez Ulloa, C. 2024. Theoretical Study on Multi-Objective Heuristic Search. *Proceedings of IJCAI*, 7021–7028.
5. Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM* 18(9): 509–517.
6. Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1(4): 301–358.
7. Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernandez Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. *Proceedings of ICAPS*, 394–403.
8. Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series 74. American Mathematical Society.